🤖 Agentic LLMOps Architecture for JNX-OS

Vision: Self-healing, AI-supervised enterprise SaaS platform

Status: 🚧 Phase 0 - Architecture Design

📊 Executive Summary

Die Gemini-Konversation beschreibt ein **“Surgical Assistant” System** - einen autonomen KI-Agenten, der:
- ✅ Das System 24/7 überwacht
- ✅ Performance-Probleme und Bugs **proaktiv** erkennt
- ✅ Code-Fixes vorschlägt (mit Diff-View)
- ✅ **Niemals** automatisch deployed (Human-in-the-Loop)
- ✅ Dependency-aware dank CodeWiki-Integration

Marktfähigkeit: Dieses Feature ist ein **Enterprise-Verkaufsargument**:
- **B2B:** “Wartungsfreies SaaS”
- **Enterprise:** “AI-supervised Infrastructure”
- **Developer:** “DevOps-Autopilot”

🏗️ Die 3 Säulen der Self-Healing-Architektur

Säule 1: Die Augen (Observability Pipeline) 👁️

Was: Globaler Error-Handler + Performance-Tracking

Wo implementieren:

```
/home/ubuntu/jnx-os/nextjs_space/
├── lib/observability/
│   ├── logger.ts              # ✅ Bereits vorhanden
│   ├── error-tracker.ts       # 🆕 NEU: Globaler Error-Handler
│   ├── performance-monitor.ts # 🆕 NEU: Latency Tracking
│   └── health-aggregator.ts   # 🆕 NEU: System Health Score
```

Neue Supabase-Tabelle:

```sql
CREATE TABLE system_health_logs (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  log_type TEXT NOT NULL,           -- 'error', 'performance', 'warning'
  severity TEXT NOT NULL,           -- 'low', 'medium', 'high', 'critical'
  component TEXT NOT NULL,          -- 'auth', 'database', 'api', 'webhook'
  error_message TEXT,
  stack_trace TEXT,
  execution_time_ms INTEGER,        -- für Performance-Tracking
  request_path TEXT,                -- API Route oder Page
  user_id UUID,                     -- Optional: Betroffener User
  org_id UUID,                      -- Optional: Betroffene Org
  metadata JSONB,                   -- Flexibel: Request Headers, Query Params, etc.
  resolved_at TIMESTAMPTZ,          -- Wann wurde das Problem behoben?
  created_at TIMESTAMPTZ DEFAULT NOW()
);

CREATE INDEX idx_health_logs_severity ON system_health_logs(severity, created_at DESC)
;
CREATE INDEX idx_health_logs_component ON system_health_logs(component, created_at DES
C);
CREATE INDEX idx_health_logs_unresolved ON system_health_logs(resolved_at) WHERE re-
solved_at IS NULL;
```

Beispiel-Integration:

```
// lib/observability/error-tracker.ts
import { createSupabaseServerClient } from '@/lib/supabase/server'
import { redactSensitiveData } from '@/lib/privacy/redaction'

export interface HealthLogEntry {
  log_type: 'error' | 'performance' | 'warning'
  severity: 'low' | 'medium' | 'high' | 'critical'
  component: string
  error_message?: string
  stack_trace?: string
  execution_time_ms?: number
  request_path?: string
  user_id?: string
  org_id?: string
  metadata?: Record<string, any>
}

export async function trackHealthEvent(entry: HealthLogEntry) {
  const supabase = createSupabaseServerClient()

  // GDPR: Redact PII before logging
  const safeEntry = {
    ...entry,
    error_message: entry.error_message ? redactSensitiveData(entry.error_message) : nu
ll,
    stack_trace: entry.stack_trace ? redactSensitiveData(entry.stack_trace) : null,
    metadata: entry.metadata ? JSON.parse(redactSensitiveData(JSON.stringi-
fy(entry.metadata))) : null
  }

  const { error } = await supabase
    .from('system_health_logs')
    .insert(safeEntry)

  if (error) {
    // Fallback: Console-Log wenn DB nicht erreichbar
    console.error('[HealthTracker] Failed to log:', error)
  }

  // Critical Errors → Sofortiges Alert (später: Slack/Email)
  if (entry.severity === 'critical') {
    console.error('[CRITICAL]', entry.component, entry.error_message)
    // TODO Phase 2: await sendSlackAlert(entry)
  }
}

// Global Error Boundary Wrapper
export function withErrorTracking<T extends Function>(fn: T, component: string): T {
  return (async (...args: any[]) => {
    const startTime = Date.now()
    try {
      const result = await fn(...args)
      const executionTime = Date.now() - startTime

      // Track slow operations
      if (executionTime > 2000) {
        await trackHealthEvent({
          log_type: 'performance',
          severity: 'medium',
          component,
          execution_time_ms: executionTime,
          metadata: { function: fn.name, args_length: args.length }
```

```
        })
      }

      return result
    } catch (error: any) {
      await trackHealthEvent({
        log_type: 'error',
        severity: 'high',
        component,
        error_message: error.message,
        stack_trace: error.stack,
        metadata: { function: fn.name }
      })
      throw error
    }
  }) as T
}
```

Säule 2: Das Gehirn (Background Observer) 🧠

Was: Cron-Job mit Gemini/Abacus AI, der Fehler analysiert

Implementierung:

```
/home/ubuntu/jnx-os/nextjs_space/
├── app/api/agentic/
│   ├── analyze-health/route.ts     # 🆕 Cron-Endpoint
│   ├── generate-proposal/route.ts  # 🆕 KI generiert Fix
│   └── apply-fix/route.ts          # 🆕 GitHub Commit Trigger
├── lib/agentic/
│   ├── health-analyzer.ts          # 🆕 Gemini-Integration
│   ├── code-context.ts             # 🆕 CodeWiki Reader
│   └── github-committer.ts         # 🆕 Git Operations
```

Cron-Job Setup (Vercel Cron):

```
// vercel.json
{
  "crons": [
    {
      "path": "/api/agentic/analyze-health",
      "schedule": "0 */3 * * *"  // Alle 3 Stunden
    }
  ]
}
```

Health Analyzer mit Gemini:

```typescript
// lib/agentic/health-analyzer.ts
import { createSupabaseServerClient } from '@/lib/supabase/server'

export interface HealthAnalysis {
  critical_issues: {
    component: string
    error_message: string
    frequency: number
    first_seen: string
    last_seen: string
  }[]
  performance_bottlenecks: {
    request_path: string
    avg_execution_time: number
    p95_execution_time: number
  }[]
  recommendations: string[]
}

export async function analyzeSystemHealth(): Promise<HealthAnalysis> {
  const supabase = createSupabaseServerClient()

  // 1. Fetch unresolved critical issues (last 24h)
  const { data: criticalIssues } = await supabase
    .from('system_health_logs')
    .select('*')
    .eq('severity', 'critical')
    .is('resolved_at', null)
    .gte('created_at', new Date(Date.now() - 24 * 60 * 60 * 1000).toISOString())
    .order('created_at', { ascending: false })

  // 2. Fetch performance issues (execution_time > 2s, last 7 days)
  const { data: perfIssues } = await supabase
    .from('system_health_logs')
    .select('request_path, execution_time_ms')
    .eq('log_type', 'performance')
    .gte('created_at', new Date(Date.now() - 7 * 24 * 60 * 60 * 1000).toISOString())

  // 3. Aggregate & Group
  const groupedErrors = new Map<string, any[]>()
  criticalIssues?.forEach(issue => {
    const key = `${issue.component}:${issue.error_message}`
    if (!groupedErrors.has(key)) groupedErrors.set(key, [])
    groupedErrors.get(key)!.push(issue)
  })

  const groupedPerf = new Map<string, number[]>()
  perfIssues?.forEach(issue => {
    if (!groupedPerf.has(issue.request_path)) groupedPerf.set(issue.request_path, [])
    groupedPerf.get(issue.request_path)!.push(issue.execution_time_ms)
  })

  // 4. Build Analysis
  const critical_issues = Array.from(groupedErrors.entries()).map(([key, issues]) => (
{
    component: issues[0].component,
    error_message: issues[0].error_message,
    frequency: issues.length,
    first_seen: issues[issues.length - 1].created_at,
    last_seen: issues[0].created_at
  }))
```

```
  const performance_bottlenecks = Array.from(groupedPerf.entries()).map(([path, times]
) => {
    const sorted = times.sort((a, b) => a - b)
    return {
      request_path: path,
      avg_execution_time: Math.round(times.reduce((a, b) => a + b, 0) / times.length),
      p95_execution_time: sorted[Math.floor(sorted.length * 0.95)]
    }
  }).filter(b => b.avg_execution_time > 2000) // Nur > 2s

  return {
    critical_issues,
    performance_bottlenecks,
    recommendations: [] // Wird von Gemini befüllt
  }
}

// Gemini Integration (nutzt Abacus.AI API)
export async function generateAIRecommendations(
  analysis: HealthAnalysis,
  codeContext: string // CodeWiki-Kontext
): Promise<string[]> {
  const prompt = `
Du bist ein Expert DevOps Engineer für ein Next.js 14 + Clerk + Supabase SaaS.

**System Health Analysis:**
${JSON.stringify(analysis, null, 2)}

**Code Context (relevant files):**
${codeContext}

**Task:**
1. Analysiere die kritischen Fehler und Performance-Probleme
2. Identifiziere Root Causes
3. Schlage konkrete Code-Fixes vor (mit Dateinamen und Diff)
4. Bewerte Impact (1-10) und Komplexität (low/medium/high)

**Output Format:**
JSON Array mit:
{
  "issue_type": "bug" | "performance" | "security",
  "component": string,
  "description": string,
  "suggested_fix": {
    "file_path": string,
    "old_code": string,
    "new_code": string,
    "explanation": string
  },
  "impact_score": number (1-10),
  "complexity": "low" | "medium" | "high"
}
`

  // Abacus.AI API Call
  const response = await fetch('https://routellm.abacus.ai/v1/chat/completions', {
    method: 'POST',
    headers: {
      'Content-Type': 'application/json',
      'Authorization': `Bearer ${process.env.ABACUSAI_API_KEY}`
    },
    body: JSON.stringify({
      model: 'gemini/gemini-2.0-flash-exp', // 2M Token Context Window!
```

```
      messages: [{ role: 'user', content: prompt }],
      response_format: { type: 'json_object' },
      temperature: 0.3 // Niedrig für deterministische Code-Vorschläge
    })
  })

  const data = await response.json()
  return JSON.parse(data.choices[0].message.content)
}
```

Säule 3: Die Hände (Proposal System) 🤲

Was: KI-Vorschläge speichern + Human-Approval-Workflow

Neue Supabase-Tabelle:

```
CREATE TABLE ai_proposals (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  module_name TEXT,              -- 'auth', 'database', 'api/webhooks/clerk'
  issue_type TEXT NOT NULL,      -- 'bug', 'performance', 'security'
  description TEXT NOT NULL,
  file_path TEXT NOT NULL,       -- z.B. 'lib/db/helpers.ts'
  old_code TEXT,                 -- Original Code
  new_code TEXT NOT NULL,        -- AI-optimierter Code
  explanation TEXT NOT NULL,     -- Warum ist das besser?
  impact_score INTEGER CHECK (impact_score BETWEEN 1 AND 10),
  complexity TEXT CHECK (complexity IN ('low', 'medium', 'high')),
  status TEXT DEFAULT 'pending' CHECK (status IN ('pending', 'approved', 'rejected', '
applied')),

  -- Audit Trail
  created_at TIMESTAMPTZ DEFAULT NOW(),
  reviewed_by UUID REFERENCES users(id),
  reviewed_at TIMESTAMPTZ,
  applied_at TIMESTAMPTZ,
  github_commit_sha TEXT,        -- Commit Hash nach Apply

  -- Relations
  related_health_log_ids UUID[]  -- Welche Health Logs haben zu diesem Proposal ge-
führt?
);

CREATE INDEX idx_proposals_status ON ai_proposals(status, created_at DESC);
CREATE INDEX idx_proposals_impact ON ai_proposals(impact_score DESC, created_at DESC);
```

API Route für Cron-Job:

```typescript
// app/api/agentic/analyze-health/route.ts
import { NextResponse } from 'next/server'
import { createSupabaseServerClient } from '@/lib/supabase/server'
import { analyzeSystemHealth, generateAIRecommendations } from '@/lib/agentic/health-analyzer'

export const dynamic = 'force-dynamic'
export const maxDuration = 300 // 5 Minuten für Gemini API

export async function GET() {
  // SECURITY: Verify Cron Secret (Vercel Cron Auth)
  const authHeader = request.headers.get('authorization')
  if (authHeader !== `Bearer ${process.env.CRON_SECRET}`) {
    return NextResponse.json({ error: 'Unauthorized' }, { status: 401 })
  }

  try {
    // 1. Analyze System Health
    const analysis = await analyzeSystemHealth()

    // 2. Load Code Context (simplified - später CodeWiki)
    const codeContext = await loadRelevantCodeFiles(analysis)

    // 3. Generate AI Recommendations
    const recommendations = await generateAIRecommendations(analysis, codeContext)

    // 4. Store as Proposals
    const supabase = createSupabaseServerClient()
    const proposals = recommendations.map(rec => ({
      module_name: rec.component,
      issue_type: rec.issue_type,
      description: rec.description,
      file_path: rec.suggested_fix.file_path,
      old_code: rec.suggested_fix.old_code,
      new_code: rec.suggested_fix.new_code,
      explanation: rec.suggested_fix.explanation,
      impact_score: rec.impact_score,
      complexity: rec.complexity
    }))

    const { data, error } = await supabase
      .from('ai_proposals')
      .insert(proposals)
      .select()

    if (error) throw error

    return NextResponse.json({
      success: true,
      proposals_created: data.length,
      critical_issues: analysis.critical_issues.length,
      performance_issues: analysis.performance_bottlenecks.length
    })

  } catch (error: any) {
    console.error('[AgenticAnalyzer] Error:', error)
    return NextResponse.json(
      { error: error.message },
      { status: 500 }
    )
  }
}
```

```
async function loadRelevantCodeFiles(analysis: HealthAnalysis): Promise<string> {
  // TODO Phase 2: CodeWiki Integration
  // Für jetzt: Lade relevante Files basierend auf component names
  return `
// Simplified Context - später ersetzt durch CodeWiki
// File: lib/db/helpers.ts
export async function upsertUser(...) { ... }

// File: app/api/webhooks/clerk/route.ts
export async function POST(...) { ... }
  `.trim()
}
```

🎨 Admin Dashboard: "AI Advisor" Panel

Wo: `/admin/ai-advisor`

UI-Components (JNX Dark Design):

```
/home/ubuntu/jnx-os/nextjs_space/
├── app/admin/ai-advisor/
│   ├── page.tsx                # 🆕 Main Dashboard
│   └── ai-advisor-client.tsx   # 🆕 Interactive UI
├── components/admin/
│   ├── proposal-card.tsx       # 🆕 Card mit Diff-View
│   ├── code-diff-viewer.tsx    # 🆕 Side-by-Side Diff
│   └── apply-fix-button.tsx    # 🆕 GitHub Commit Trigger
```

Mockup-Struktur:

```
// app/admin/ai-advisor/page.tsx
import { requireAdmin } from '@/lib/auth/helpers'
import { createSupabaseServerClient } from '@/lib/supabase/server'
import AIAdvisorClient from './ai-advisor-client'

export const dynamic = 'force-dynamic'

export default async function AIAdvisorPage() {
  const { user, jnxUser } = await requireAdmin()
  const supabase = createSupabaseServerClient()

  // Fetch Pending Proposals
  const { data: proposals } = await supabase
    .from('ai_proposals')
    .select('*')
    .eq('status', 'pending')
    .order('impact_score', { ascending: false })
    .order('created_at', { ascending: false })

  return <AIAdvisorClient user={user} proposals={proposals || []} />
}
```

AI Advisor Client Component:

```
// app/admin/ai-advisor/ai-advisor-client.tsx
'use client'

import { useState } from 'react'
import { AlertTriangle, CheckCircle2, Clock, Zap } from 'lucide-react'
import { ProposalCard } from '@/components/admin/proposal-card'
import { StatusBadge } from '@/components/ui/status-badge'

export default function AIAdvisorClient({ user, proposals }) {
  const [activeProposal, setActiveProposal] = useState<string | null>(null)

  const criticalProposals = proposals.filter(p => p.impact_score >= 8)
  const highProposals = proposals.filter(p => p.impact_score >= 5 && p.impact_score <
8)
  const mediumProposals = proposals.filter(p => p.impact_score < 5)

  return (
    <div className="min-h-screen bg-jnx-darker">
      {/* Header */}
      <div className="border-b border-slate-800 bg-jnx-dark">
        <div className="max-w-7xl mx-auto px-6 py-6">
          <div className="flex items-center justify-between">
            <div>
              <h1 className="text-2xl font-bold text-white flex items-center gap-3">
                <Zap className="text-cyan-500" size={28} />
                AI Advisor Dashboard
              </h1>
              <p className="text-slate-400 mt-1">
                AI-generierte Optimierungsvorschläge für JNX-OS
              </p>
            </div>
            <div className="flex gap-3">
              <StatusBadge status="connected">
                Gemini 2.0 Active
              </StatusBadge>
              <StatusBadge status="online">
                {proposals.length} Pending
              </StatusBadge>
            </div>
          </div>
        </div>
      </div>

      {/* Stats */}
      <div className="max-w-7xl mx-auto px-6 py-6">
        <div className="grid grid-cols-1 md:grid-cols-3 gap-4 mb-8">
          <div className="bg-slate-900/40 border border-red-500/30 rounded-xl p-4">
            <div className="flex items-center justify-between">
              <div>
                <p className="text-slate-400 text-sm">Critical Issues</p>
                <p className="text-3xl font-bold text-red-400 mt-1">
                  {criticalProposals.length}
                </p>
              </div>
              <AlertTriangle className="text-red-400" size={32} />
            </div>
          </div>

          <div className="bg-slate-900/40 border border-yellow-500/30 rounded-xl p-4">
            <div className="flex items-center justify-between">
              <div>
                <p className="text-slate-400 text-sm">High Priority</p>
```

```
                <p className="text-3xl font-bold text-yellow-400 mt-1">
                  {highProposals.length}
                </p>
              </div>
              <Clock className="text-yellow-400" size={32} />
            </div>
          </div>

          <div className="bg-slate-900/40 border border-cyan-500/30 rounded-xl p-4">
            <div className="flex items-center justify-between">
              <div>
                <p className="text-slate-400 text-sm">Optimizations</p>
                <p className="text-3xl font-bold text-cyan-400 mt-1">
                  {mediumProposals.length}
                </p>
              </div>
              <CheckCircle2 className="text-cyan-400" size={32} />
            </div>
          </div>
        </div>

        {/* Proposals List */}
        <div className="space-y-4">
          {criticalProposals.length > 0 && (
            <div>
              <h2 className="text-xl font-bold text-white mb-4 flex items-center
gap-2">
                <AlertTriangle className="text-red-400" size={20} />
                Critical Issues
              </h2>
              {criticalProposals.map(proposal => (
                <ProposalCard
                  key={proposal.id}
                  proposal={proposal}
                  isActive={activeProposal === proposal.id}
                  onToggle={() => setActiveProposal(
                    activeProposal === proposal.id ? null : proposal.id
                  )}
                />
              ))}
            </div>
          )}

          {/* Repeat for highProposals and mediumProposals */}
        </div>
      </div>
    </div>
  )
}
```

📈 Implementation Roadmap

Phase 0: Foundation (Week 1-2) 🏗️

- [x] Architektur-Dokument erstellt
- [ ] Database Schema Migration (`system_health_logs` , `ai_proposals`)
- [ ] Error Tracker implementieren (`lib/observability/error-tracker.ts`)
- [ ] Health Analyzer Basis-Implementierung

- [ ] Admin Dashboard Mockup

Phase 1: Observability (Week 3-4) 👁️

- [ ] Global Error Boundary in allen API Routes
- [ ] Performance Monitoring für langsame Queries (>2s)
- [ ] Health Logs Dashboard in `/admin`
- [ ] GDPR-compliant PII Redaction

Phase 2: AI Integration (Week 5-7) 🧠

- [ ] Gemini 2.0 Integration (via Abacus.AI)
- [ ] Cron-Job Setup (Vercel Cron)
- [ ] Code Context Loader (später: CodeWiki)
- [ ] AI Proposal Generator
- [ ] Proposal Storage in DB

Phase 3: Human-in-the-Loop (Week 8-9) 🤲

- [ ] AI Advisor Dashboard UI (`/admin/ai-advisor`)
- [ ] Proposal Card Component mit Diff-View
- [ ] GitHub Integration (Auto-Commit)
- [ ] Approval Workflow (approve/reject)
- [ ] Rollback-Mechanismus

Phase 4: Production Hardening (Week 10-12) 🛡️

- [ ] Rate Limiting für AI API Calls
- [ ] Audit Logging für AI-Proposals
- [ ] Slack/Email Alerts für Critical Issues
- [ ] A/B Testing für AI-Fixes
- [ ] Monitoring & Alerting

🔒 Security & Compliance

GDPR Compliance:

- ✅ PII Redaction in allen Health Logs
- ✅ Kein automatisches Deployment ohne Human-Approval
- ✅ Audit Trail für alle AI-Actions

Security:

- ✅ Cron-Endpoint mit Secret-Auth
- ✅ Admin-only Access für AI Advisor Dashboard
- ✅ Code-Diffs werden **niemals** public exposed
- ✅ GitHub Commits via Personal Access Token (encrypted)

💰 Monetization Strategy

JNX-OS Tier:

- **Free:** Nur Health Logs (kein AI)
- **Pro:** 10 AI Proposals/Monat
- **Enterprise:** Unlimited + Priority Support

QRYX Integration:

- **Add-On Feature:** “AI-Supervised Data Pipeline”
- **Value Proposition:** “Your data pipeline optimizes itself”
- **Pricing:** +$99/mo für QRYX Enterprise Kunden

📚 References

- **Gemini 2.0 API:** https://ai.google.dev/gemini-api
- **Abacus.AI RouteLLM:** https://abacus.ai/app/route-llm-apis
- **Vercel Cron Jobs:** https://vercel.com/docs/cron-jobs
- **CodeWiki (Future):** TBD

Status: 🚧 Ready for Implementation

Next Steps:
1. Review dieses Dokument
2. Database Migration ausführen
3. Error Tracker implementieren
4. MVP des AI Advisor Dashboards bauen

Version: 1.0
Last Updated: 2024-12-28
Author: DeepAgent + User Collaboration
Project: JNX-OS Agentic LLMOps